

Informe técnico: SistemaComercialUPLA – Semana 3

Aplicación web con dos actividades de cálculo comercial (Actividad 01: Tienda · Actividad 02: Transporte)

1. Resumen

El proyecto **SistemaComercialUPLA** es una aplicación web empaquetada como WAR con Maven. Contiene dos páginas JSP que implementan calculadoras de negocio con reglas de descuento y obsequios:

- **Actividad 01 (`index.jsp`)**: control de ventas de una tienda. Calcula el importe de la compra, el descuento según docenas adquiridas, el importe a pagar y los lapiceros de obsequio.
- **Actividad 02 (`transporte.jsp`)**: venta de pasajes en la ruta Lima – Huancayo. Calcula el importe de la compra, el descuento según cantidad de pasajes, el importe a pagar y los caramelos de obsequio.

Toda la lógica de cálculo se ejecuta en el navegador mediante JavaScript; el servidor únicamente entrega las páginas.

2. Estructura del proyecto

Archivo	Función
<code>pom.xml</code>	Configuración Maven: empaquetado WAR, Java 17, dependencia Jakarta EE 10 (scope <i>provided</i>), <code>maven-war-plugin</code> 3.4.0.
<code>src/main/webapp/index.jsp</code>	Actividad 01: Control de Ventas, Descuentos y Obsequios.
<code>src/main/webapp/transporte.jsp</code>	Actividad 02: Ruta Lima – Huancayo, Pasajes y Obsequios.
<code>src/main/webapp/css/estilos.css</code>	Estilos compartidos (tarjetas, botón principal, tipografía).
<code>src/main/webapp/WEB-INF/web.xml</code>	Descriptor de despliegue (Jakarta EE, versión 6.0); página de inicio <code>index.jsp</code> .

No existen clases Java (carpeta `src/main/java`), servlets ni pruebas automatizadas.

3. Tecnologías

- **Java 17 y Maven** (`groupId: com.upla`, `artifactId: SistemaComercialUPLA`, versión `1.0-SNAPSHOT`).
- **Jakarta EE 10** (`jakarta.jakartaee-web-api 10.0.0`, *provided*). Requiere un contenedor compatible con Servlet 6.0, por ejemplo Apache Tomcat 10.1+, GlassFish 7 o WildFly 27+.
- **JSP** como plantilla de las vistas, con `isELIgnored="true"`.
- **Tailwind CSS** cargado desde CDN (`cdn.tailwindcss.com`) más una hoja de estilos propia.
- **JavaScript** nativo para la lógica de negocio.

4. Actividad 01 – Tienda (`index.jsp`)

4.1 Entradas

- Precio de la docena (S/.), valor inicial 45.00.
- Cantidad de docenas adquiridas, valor inicial 12.

4.2 Reglas de negocio implementadas

Concepto	Regla
Importe de la compra	precio × docenas
Descuento	20 % si docenas ≥ 10; en caso contrario 10 %
Importe a pagar	importe de la compra – descuento
Lapiceros de obsequio	2 por cada docena si el importe a pagar ≥ S/. 200; si no, 0

4.3 Ejemplo con los valores por defecto

12 docenas a S/. 45.00: importe S/. 540.00 → descuento S/. 108.00 (20 %) → a pagar S/. 432.00 → **24 lapiceros**. Estos valores coinciden con los que se muestran precargados en la página.

4.4 Casos de prueba (calculados a partir del código)

Docenas	Importe	Descuento	A pagar	Lapiceros
12	540.00	108.00 (20 %)	432.00	24
10	450.00	90.00 (20 %)	360.00	20
5	225.00	22.50 (10 %)	202.50	10
4	180.00	18.00 (10 %)	162.00	0

5. Actividad 02 – Transporte (`transporte.jsp`)

5.1 Entradas

- Turno del viaje: Mañana o Noche (ambos a S/. 37.50).
- Cantidad de pasajes adquiridos, valor inicial 16.

5.2 Reglas de negocio implementadas

Concepto	Regla
Precio del pasaje	S/. 37.50 fijo
Importe de la compra	pasajes × 37.50
Descuento	8 % si pasajes ≥ 15; en caso contrario 5 %
Importe a pagar	importe de la compra – descuento
Caramelos de obsequio	2 por boleto si el importe a pagar > S/. 200; si no, 0

5.3 Ejemplo con los valores por defecto

16 pasajes: importe S/. 600.00 → descuento S/. 48.00 (8 %) → a pagar S/. 552.00 → **32 caramelos**. Coincide con los valores precargados en la página.

5.4 Casos de prueba (calculados a partir del código)

Pasajes	Importe	Descuento	A pagar	Caramelos
16	600.00	48.00 (8 %)	552.00	32

Pasajes	Importe	Descuento	A pagar	Caramelos
15	562.50	45.00 (8 %)	517.50	30
14	525.00	26.25 (5 %)	498.75	28
6	225.00	11.25 (5 %)	213.75	12
4	150.00	7.50 (5 %)	142.50	0

6. Diseño de la interfaz

Ambas páginas comparten la misma estructura: encabezado con título y navegación entre actividades, un panel izquierdo de datos de entrada y un panel derecho de resultados en tarjetas de color (neutra, azul, verde y un color de obsequio: morado en la Actividad 01 y ámbar en la Actividad 02). El diseño es responsivo (una columna en pantallas pequeñas y dos columnas desde lg). La hoja `estilos.css` define las clases `card-white`, `card-blue` y `btn-primary-upla` (azul #1A66FF), reutilizadas en ambas vistas.

7. Observaciones y hallazgos

- 1. El turno no influye en el cálculo.** En `transporte.jsp` el selector `turnoSelect` nunca se lee en el script. Como ambos turnos cuestan S/. 37.50 el resultado no cambia, pero si el enunciado exige tarifas distintas por turno, falta implementarlo.
- 2. Criterio de frontera distinto para los obsequios.** La Actividad 01 usa `importePagar >= 200` y la Actividad 02 usa `importePagar > 200` (el texto de la interfaz también lo indica así). Conviene confirmar con el enunciado que ambas condiciones son las correctas.
- 3. Sin validación de entradas.** Los campos numéricos usan `step="any"` y no tienen `min`; se aceptan cantidades negativas o decimales (por ejemplo, 16.5 pasajes). Un valor vacío se trata como 0.
- 4. Sin lógica en el servidor.** Aunque el proyecto es JSP/Jakarta EE, no hay servlets ni clases Java. Si la asignatura requiere separar capas (modelo, controlador, vista), esta parte está pendiente.
- 5. Dependencia de Internet.** Tailwind se carga desde un CDN, por lo que sin conexión la interfaz pierde sus estilos. Además, el propio CDN indica que no es apropiado para producción.
- 6. Resultados solo al pulsar el botón.** Los resultados no se recalculan al cambiar los campos; hasta pulsar el botón se muestran los valores iniciales fijos del HTML.

7. **Código duplicado.** El encabezado, el menú y la estructura de las tarjetas se repiten en ambas páginas; podrían extraerse a un fragmento JSP común (`<jsp:include>`).
8. **Coherencia técnica correcta.** El `web.xml` (versión 6.0) y la dependencia Jakarta EE 10 son consistentes entre sí, y Java 17 es compatible con ellos.

8. Recomendaciones

- Usar el valor del turno en el cálculo, o documentar que la tarifa es única.
- Añadir `min="0"` y `step="1"` a las cantidades, y validar en el script.
- Verificar con el enunciado los umbrales (≥ 15 , ≥ 10 , ≥ 200 , > 200).
- Si se requiere Java del lado del servidor, mover los cálculos a una clase de negocio y un servlet.
- Incluir Tailwind compilado localmente (o prescindir de él) para funcionar sin conexión.
- Agregar pruebas con los casos de las secciones 4.4 y 5.4.

9. Cómo ejecutar el proyecto

1. Compilar: `mvn clean package`. Se genera `target/SistemaComercialUPLA.war`.
2. Desplegar el WAR en un contenedor compatible con Jakarta EE 10 (por ejemplo, Tomcat 10.1).
3. Abrir `http://localhost:8080/SistemaComercialUPLA/` (la página inicial es `index.jsp`) y navegar a la Actividad 02 desde el menú superior.

10. Conclusión

El proyecto cumple su objetivo: ambas actividades calculan correctamente importes, descuentos y obsequios según las reglas implementadas, y las interfaces son claras y consistentes. Los puntos a revisar son sobre todo el uso del turno, los criterios de frontera, la validación de datos y la ausencia de lógica en el servidor, que dependerán de lo que exija el enunciado de la actividad.

Nota: este informe se basa únicamente en los archivos del ZIP; las reglas de negocio se infirieron del código, ya que el enunciado original no estaba incluido.